Extending Maximum Likelihood Reinforcement Learning to Continuous Rewards via Reward-Mass Normalization

Sungi Hong, Junyu Leng, Qianhui Ye
Texas A&M Univesrsity
{sungih2,levileng,cindyye49}@tamu.edu

Abstract

Standard policy-gradient methods optimize expected return, which corresponds to a first-order approximation of the maximum likelihood (ML) objective. Maximum Likelihood Reinforcement Learning (MaxRL) bridges this gap by constructing a family of objectives that progressively tighten this approximation as rollout compute increases. However, MaxRL is formulated exclusively for binary-reward settings, limiting its applicability to the broad class of problems with continuous or dense reward signals. We propose *Continuous-Reward MaxRL* (CR-MaxRL), which replaces the success-count normalization of MaxRL with reward-mass normalization. This yields a reward-weighted policy-gradient estimator that scales each trajectory's contribution by its return relative to the batch, preserving the likelihood-inspired normalization principle while accommodating arbitrary reward scales. To mitigate the risk of premature policy collapse onto a small set of high-reward trajectories, we further introduce an entropy-regularized variant that balances exploitation of high-return behaviors with sufficient exploration diversity. We evaluate CR-MaxRL on LunarLander-v3, Acrobot-v1, Hopper-v5, Walker2d-v5, and the GSM8K mathematical reasoning benchmark. Across environments, CR-MaxRL improves training stability over standard REINFORCE and substantially outperforms binary MaxRL in settings where exact successes are sparse. Properly tuned entropy regularization further improves performance and cross-seed robustness. These results demonstrate that likelihood-inspired reward normalization generalizes naturally beyond binary settings, providing a principled and broadly applicable policy-gradient framework.

1 Introduction

Reinforcement learning (RL) has become a central paradigm for training agents in environments with non-differentiable objectives and sparse feedback [Sutton et al.]. In reasoning-oriented tasks, such as mathematical problem-solving or code generation, the ultimate objective is to maximize the probability that a sampled output is correct. Classical policy-gradient algorithms, such as REINFORCE [Williams, 1992] and its modern variants, instead optimize expected reward, which in binary settings equals the expected pass rate (pass@1). Recent analysis shows that this objective is only a first-order approximation of the true maximum likelihood (ML) objective, which accounts for higher-order correctness events and places greater emphasis on rare but informative successes [Tajwar et al., 2026].

Maximum Likelihood Reinforcement Learning (MaxRL) [Tajwar et al., 2026] addresses this mismatch by constructing a compute-indexed family of objectives that progressively tighten the approximation to the ML objective as the number of sampled rollouts grows. In binary-reward settings, MaxRL normalizes the policy-gradient update by the number of successful trajectories in each batch, rather

2026 Spring ECEN 743 Final Project.

than by the total batch size. This weighting concentrates learning signal on positive outcomes and yields improved scaling behavior: additional rollout compute not only reduces gradient variance but also improves the fidelity of the optimized objective itself.

Despite these advantages, existing MaxRL formulations are restricted to binary rewards, where each trajectory is labeled either successful or unsuccessful. This restriction significantly limits applicability, as many practical RL problems, including continuous control and partial-credit reasoning tasks, involve dense or real-valued reward signals. Mapping continuous rewards to binary indicators discards valuable information about the relative quality of trajectories and can lead to suboptimal learning dynamics, particularly when successes are initially absent from the rollout batch.

We propose to extend MaxRL to continuous-reward settings by reinterpreting the normalization in the binary estimator. Instead of normalizing by the count of successful trajectories, we normalize by the total reward mass in the batch. This yields a trajectory weighting scheme in which each sample contributes to the gradient in proportion to its return relative to other trajectories in the batch. The resulting estimator, which we call CR-MaxRL, preserves the core insight of MaxRL while enabling richer feedback signals. To handle negative returns and large reward scales, we apply a min-shift transformation that maps batch returns to nonnegative weights without altering their relative ordering.

Reward-mass normalization introduces a potential failure mode: if a small subset of trajectories accumulates most of the reward mass, the policy may prematurely collapse to a narrow set of behaviors. We address this with an entropy-regularized variant of CR-MaxRL that augments the weighted likelihood objective with a policy-entropy term. This balances exploitation of high-reward trajectories against maintaining behavioral diversity, and we show theoretically and empirically that the regularization coefficient must be carefully chosen to be beneficial.

We evaluate our approach across a diverse set of environments spanning sparse-reward locomotion (LunarLander-v3), dense-reward locomotion (Hopper-v5, Walker2d-v5), negative-reward control (Acrobot-v1), and language-model reasoning (GSM8K). CR-MaxRL consistently improves over standard REINFORCE and outperforms binary MaxRL when successes are rare, and entropy regularization with a suitable coefficient further improves both final performance and cross-seed stability.

Our contributions are as follows:

- We propose Continuous-Reward MaxRL (CR-MaxRL), extending the MaxRL framework to continuous-reward settings via reward-mass normalization with a min-shift transformation.
- We introduce an entropy-regularized variant that prevents premature policy collapse and provide a convergence analysis under standard nonconvex stochastic optimization assumptions.
- We conduct comprehensive experiments on five benchmarks, demonstrating that reward-mass normalization provides a stable and effective policy-gradient estimator across continuous-control and language-reasoning tasks.

2 Related Work

Maximum likelihood objectives in reinforcement learning. Tajwar et al. [2026] establish that standard policy-gradient methods optimize a first-order approximation to the maximum likelihood objective in binary-reward settings, and propose MaxRL as a principled interpolation between expected-reward optimization and ML training indexed by rollout compute. Our work directly extends MaxRL by relaxing the binary-reward assumption. Related to this, Davis and Recht [2025] analyze the population-level objectives implicitly optimized by binary-reward RL algorithms, providing statistical characterizations of pass-rate optimization. Xiong et al. [2025] study nonlinear objective functions based on pass rates and propose adaptive rollout allocation to improve sample efficiency. In contrast to these analyses, we focus on enabling ML-inspired normalization under continuous rewards.

Entropy regularization and maximum-entropy RL. Maximum-entropy RL [Ziebart et al., 2008] augments the return objective with a policy-entropy bonus, encouraging exploration and improving robustness. Soft Actor-Critic (SAC) [Haarnoja et al., 2018] and related methods demonstrate that entropy regularization substantially improves sample efficiency and training stability in continuous control. Our entropy-regularized CR-MaxRL incorporates the same principle within the reward-mass normalization framework, and we provide an analysis showing that the entropy coefficient

must remain below a threshold proportional to the reward-normalized gradient magnitude to avoid interfering with reward-driven learning.

Maximum likelihood in inverse reinforcement learning. While our work targets forward RL, maximum-likelihood formulations also appear prominently in inverse RL. Zeng et al. [2022] propose a maximum-likelihood IRL framework with finite-time guarantees via a single-loop stochastic gradient algorithm. Zeng et al. [2023] extend this to offline settings through a bilevel optimization framework that jointly estimates reward functions and conservative world models. Although these works share a likelihood-based perspective, they address reward recovery from demonstrations rather than policy optimization, and thus are complementary to our approach.

Reward normalization and policy-gradient variance reduction. Reward normalization is a standard technique for reducing variance in policy-gradient methods. Baseline subtraction [Williams, 1992] centers returns at zero; normalized advantage estimation [Schulman et al., 2015] further scales by the standard deviation. Our reward-mass normalization differs from these approaches in that it does not merely center or whiten the reward signal but reinterprets the normalization constant as a measure of batch-level reward mass, directly connecting the estimator to the ML objective of MaxRL. Proximal Policy Optimization (PPO) [Schulman et al., 2017] serves as a strong practical baseline; we compare against it in the continuous-control experiments.

3 Problem Formulation

We consider a reinforcement learning setting in which an agent interacts with an environment and generates trajectories $\tau \sim \pi_\theta$, where π_θ is a parameterized policy. Each trajectory τ is associated with a scalar return $R(\tau)$, which may be either binary or continuous depending on the task.

3.1 Standard Policy Gradient Objective

The classical objective in reinforcement learning is to maximize the expected return

$$J_{\mathrm{RL}}(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)].$$

Using the score function estimator, the gradient of this objective can be written as

$$\nabla J_{\mathrm{RL}}(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[R(\tau)\nabla \log \pi_\theta(\tau)\right].$$

In practice, this expectation is approximated using a Monte Carlo estimator

$$\frac{1}{N}\sum_{i=1}^{N} R_i \nabla \log \pi_\theta(\tau_i),$$

where $\{\tau_i\}_{i=1}^N$ are sampled trajectories and $R_i = R(\tau_i)$. While this formulation is general, it treats all trajectories uniformly through normalization by N, which can lead to inefficient learning in settings where high-quality trajectories are rare.

3.2 Binary MaxRL Formulation

In binary-reward settings, where $r_i \in \{0, 1\}$ indicates success or failure, Maximum Likelihood Reinforcement Learning (MaxRL) modifies the normalization by conditioning on successful trajectories. Specifically, the gradient estimator takes the form

$$\frac{1}{K}\sum_{i=1}^{N} r_i \nabla \log \pi_\theta(\tau_i),$$

where $K = \sum_{i=1}^N r_i$ is the number of successful trajectories. This formulation emphasizes successful trajectories by normalizing with respect to K rather than N, effectively increasing the contribution of rare but informative successes.

3.3 Continuous-Reward Generalization—proposed method

The binary MaxRL formulation is limited to settings where trajectories can be cleanly classified as successful or unsuccessful. However, in many RL problems, rewards are continuous and provide richer information about trajectory quality. To extend MaxRL to such settings, we reinterpret the normalization term as the total reward mass in a batch rather than the count of successful trajectories. This leads to the following reward-normalized objective

$$\nabla J_{\text{CR-MaxRL}}(\theta) = \frac{1}{\sum_{i=1}^{N} R_i} \sum_{i=1}^{N} R_i \nabla \log \pi_\theta(\tau_i).$$

In practice, rewards may be negative or exhibit large variance, which can lead to unstable updates. To address this issue, we introduce a nonnegative transformation of the rewards. Specifically, we define

$$\widetilde{R}_i = \max(R_i - \min_j R_j, 0), \quad w_i = \frac{\widetilde{R}_i}{\sum_j \widetilde{R}_j + \epsilon}.$$

Using this transformation, the gradient estimator becomes

$$\nabla J_{\text{CR-MaxRL}}(\theta) = \sum_{i=1}^{N} w_i \nabla \log \pi_\theta(\tau_i).$$

This formulation generalizes the success-based normalization of MaxRL by replacing the binary success indicator with a continuous measure of trajectory quality. The shift by $\min_j R_j$ ensures that all transformed rewards are nonnegative, while preserving the relative ordering among trajectories. As a result, all trajectories contribute to the gradient, higher-reward trajectories are emphasized, and the update remains well-defined even when no trajectory achieves high reward.

3.4 Entropy-Regularized Objective—proposed method

A potential drawback of reward-normalized formulations is that they may overly concentrate probability mass on a small set of high-reward trajectories, reducing exploration and diversity. To address this issue, we consider an entropy-regularized objective

$$J(\theta) = J_{\text{CR-MaxRL}}(\theta) + \lambda H(\pi_\theta),$$

where $H(\pi_\theta)$ denotes the entropy of the policy

$$H(\pi_\theta) = -\mathbb{E}_{\tau \sim \pi_\theta}[\log \pi_\theta(\tau)],$$

and $\lambda > 0$ is a regularization coefficient. The corresponding loss function is

$$\mathcal{L}(\theta) = -\sum_i w_i \log \pi_\theta(\tau_i) - \lambda H(\pi_\theta).$$

This formulation balances two competing objectives: maximizing the likelihood of high-reward trajectories and maintaining sufficient diversity in the policy. We evaluate several values of λ to study its effect on performance and stability.

4 Analysis on the LunarLander-v3 Environment

4.1 Overview

We adopt a standard episodic policy-gradient framework. At each iteration, we sample a batch of N trajectories $\{\tau_i\}_{i=1}^{N}$ from the current policy π_θ. Each trajectory is assigned a return $R_i = R(\tau_i)$.

Each trajectory $\tau_i = (s_{i,0}, a_{i,0}, \ldots, s_{i,T})$ is assigned a return

$$R_i = \sum_{t=0}^{T} \gamma^t r_{i,t}.$$

The policy is updated using a weighted log-likelihood objective of the form

$$\mathcal{L}(\theta) = -\sum_{i=1}^{N} w_i \log \pi_\theta(\tau_i), \quad \text{where } \log \pi_\theta(\tau_i) = \sum_t \log \pi_\theta(a_{i,t} \mid s_{i,t}).$$

We compare three policy-gradient estimators under a unified training framework:

- **Standard Policy Gradient, Sutton et al.**, which uses raw returns as weights,
- **Binary MaxRL, Tajwar et al. [2026]**, which normalizes over successful trajectories only,
- **Continuous-Reward MaxRL (proposed)**, which normalizes over continuous reward magnitudes.

4.2 Experimental Setup: LunarLander-v3

We evaluate all methods on the **LunarLander-v3** environment from Gymnasium, a standard continuous control benchmark with sparse success signals and dense intermediate rewards. Success is defined as achieving a total return greater than a predefined threshold.

Implementation Details We implement all methods using a unified policy-gradient framework with the following configuration:

- **Policy network:** two-layer MLP with Tanh activations and hidden size 128
- **Optimizer:** Adam
- **Learning rate:** 10^{-3}
- **Batch size:** 8 parallel environments
- **Discount factor:** $\gamma = 0.99$
- **Maximum episode length:** 500 steps
- **Training iterations:** 800
- **Success criterion:** An episode is considered successful if its undiscounted return exceeds a threshold of 200, i.e.,
$$\text{success}(\tau_i) = \mathbb{I}\left\{R_i^{\text{raw}} \geq 200\right\},$$
where R_i^{raw} denotes the cumulative (undiscounted) reward of trajectory τ_i.

We use parallel environment rollout via vectorized environments to improve sample efficiency. All experiments are conducted with a fixed random seed.

4.3 Results on LunarLander-v3

We evaluate the proposed methods on the LunarLander-v3 environment, which combines dense intermediate rewards with a sparse notion of success. This environment is useful for testing whether reward-normalized policy-gradient updates can provide meaningful learning signals even before successful trajectories are frequently observed. We use LunarLander-v3 as an initial testbed due to its moderate complexity, which allows for rapid experimentation across different algorithmic variants and hyperparameter settings. In particular, this setup enables us to systematically analyze the behavior of Continuous MaxRL and its entropy-regularized variant under varying configurations. We use insights from this environment, such as the impact of reward normalization and entropy regularization with different values of λ, to guide experiments on more challenging domains like Hopper, Walker2d, and GSM8K.

Average Return. Figure 1 shows the average episodic return over training iterations. Continuous MaxRL achieves the strongest overall performance among the tested methods. Compared with standard REINFORCE, Continuous MaxRL produces a smoother learning curve and reaches higher return values. This suggests that normalizing trajectory returns within each rollout batch improves the quality of the policy-gradient update by emphasizing relatively high-performing trajectories while reducing sensitivity to the absolute scale of rewards.

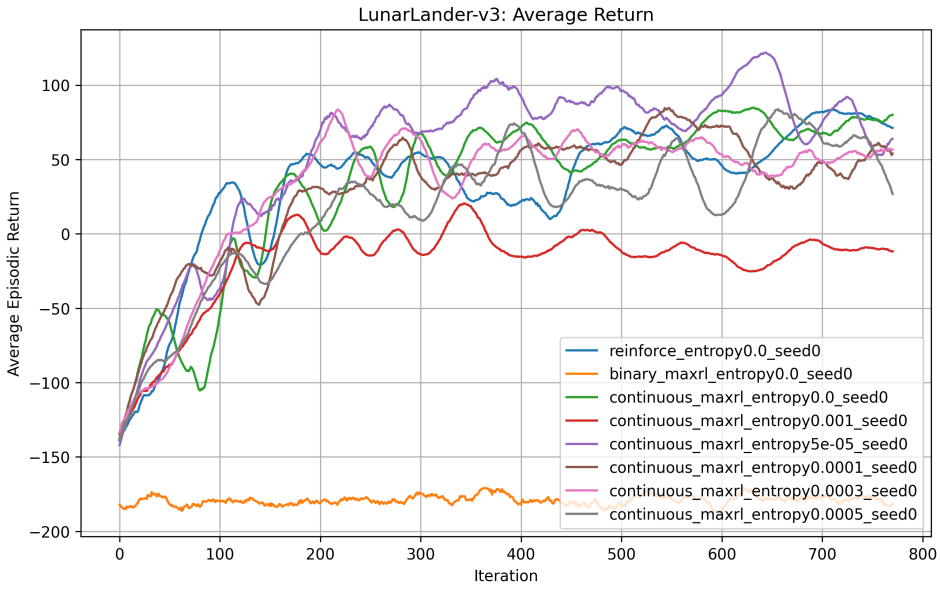

Figure 1: Average episodic return across training iterations for different methods on LunarLander-v3.

Binary MaxRL performs poorly in this setting. Since the success signal in LunarLander-v3 is sparse, most rollout batches contain no successful trajectories. In such cases, the binary MaxRL weights become zero, producing little or no useful gradient signal. This explains why Binary MaxRL fails to improve despite being effective in settings where binary successes are more frequently observed.

Success Rate. Figure 2 shows the success rate across training iterations. The results show that return improvement does not always translate directly into a high success rate. Although Continuous MaxRL improves average return substantially, success remains relatively rare under the strict success threshold. This indicates that the policy learns meaningful partial behaviors, such as improved control and landing behavior, but does not always reach the threshold required to count as a successful episode.

Among all methods, *entropy-regularized Continuous MaxRL with a small coefficient,* $\lambda = 5 \times 10^{-5}$, *achieves the most noticeable improvement in success rate.* This suggests that a small amount of entropy regularization improves exploration and helps the policy discover successful trajectories more effectively.

Policy Entropy. Figure 3 shows the policy entropy over training. REINFORCE maintains relatively high entropy, but its return performance is unstable. Continuous MaxRL achieves a better balance between exploration and exploitation, leading to more stable learning. However, excessive entropy regularization can degrade performance by preventing the policy from becoming sufficiently focused on high-return actions.

Overall, the experiments support the main hypothesis of this work: continuous reward normalization provides a denser and more stable learning signal than binary success normalization, especially in environments where successful trajectories are initially rare. In addition, a small amount of entropy regularization can further improve exploration, while overly strong entropy regularization may reduce final performance.

4.4 Effect of Entropy Coefficient

We further evaluate entropy-regularized Continuous MaxRL on the LunarLander-v3 environment by sweeping over the entropy coefficients

$$\lambda \in \{5 \times 10^{-5}, 10^{-4}, 3 \times 10^{-4}, 5 \times 10^{-4}, 10^{-3}\}.$$

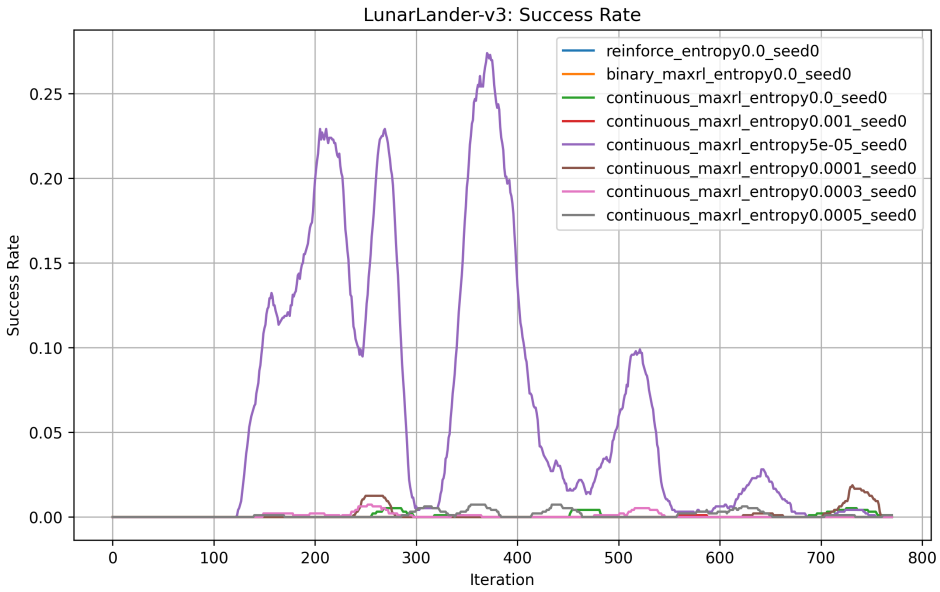

Figure 2: Success rate across training iterations on LunarLander-v3.

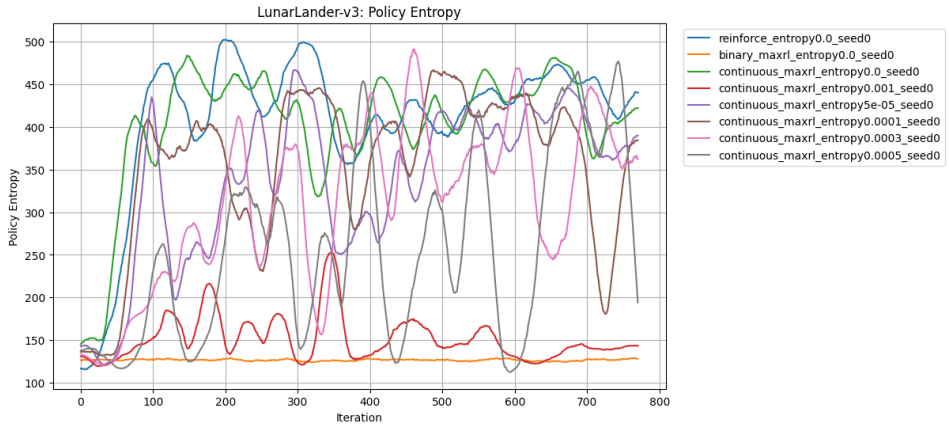

Figure 3: Policy entropy over training on LunarLander-v3.

Among the tested values, $\lambda = 5 \times 10^{-5}$ achieves the best overall performance. It obtains the highest average return, the highest final-window average return, and the largest observed success rate during training. In particular, this setting reaches a maximum success rate of 0.4375, whereas the other entropy coefficients produce substantially smaller success rates. This suggests that a small amount of entropy regularization can help the policy explore useful behaviors that are difficult to discover using reward normalization alone.

However, the benefit of entropy regularization is highly sensitive to the coefficient. When λ is increased to 10^{-4}, 3×10^{-4}, or 5×10^{-4}, the method still achieves moderate returns but does not produce comparable success rates. When $\lambda = 10^{-3}$, performance degrades significantly, resulting in negative average returns and almost no successful episodes. Thus, excessive entropy regularization does not improve exploration in this setting; instead, it appears to interfere with effective policy improvement.

The entropy curves further indicate that larger entropy coefficients do not necessarily lead to better sustained exploration. In fact, some larger values result in unstable or collapsed entropy behavior. Therefore, entropy regularization should be interpreted as a useful but sensitive stabilizing mechanism.

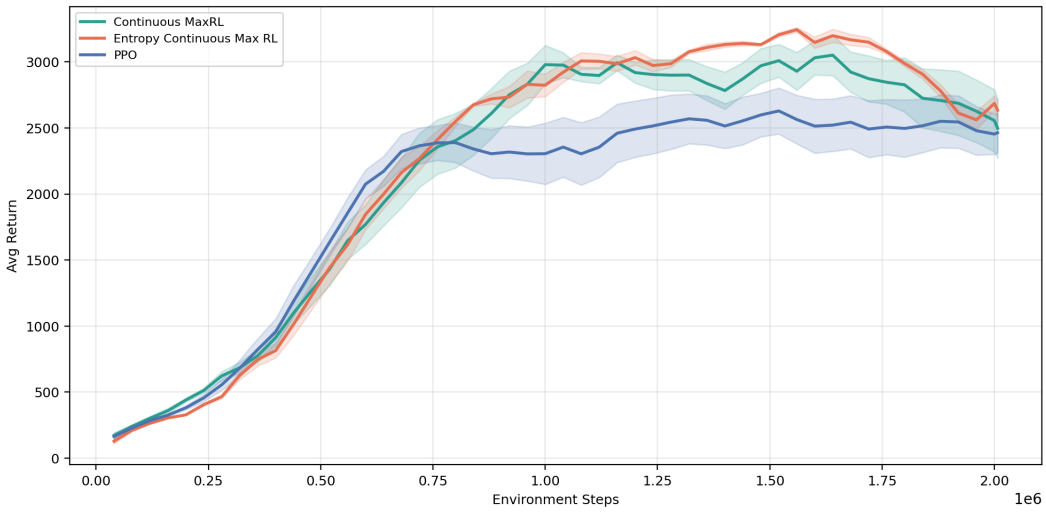

Figure 4: Return comparison on Hopper-v5 with 2000k training steps.

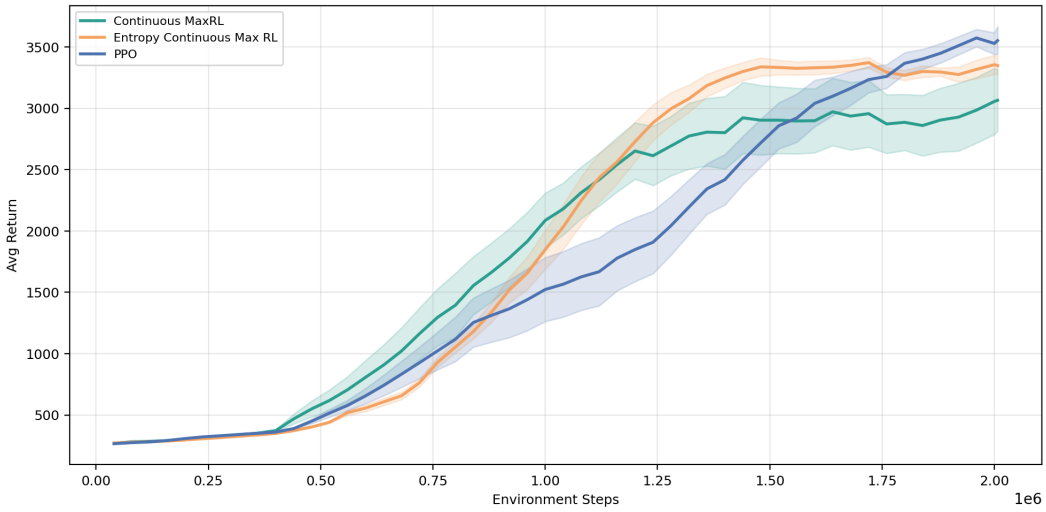

Figure 5: Return comparison on Walker2d-v5 with 2000k training steps.

The best result occurs in a narrow range where entropy is strong enough to encourage exploration but not so strong that it disrupts reward-driven learning.

These findings suggest that entropy-regularized Continuous MaxRL can improve performance, but only when the entropy coefficient is carefully tuned. A theoretical justification of this behavior is provided in Appendix A.

5 Analysis on the Hopper and Walker Environments

Having validated the effectiveness of our approach on LunarLander-v3 under a relatively short training horizon, we now consider more challenging continuous-control environments with substantially longer training steps.

Figure 4 shows that, on Hopper-v5, the best-performing method is Entropy Continuous MaxRL with an entropy coefficient of 0.0005. This method achieves an average return of 2889.49 ± 391.30 over three seeds, which is higher than both PPO (2452.39 ± 900.18) and Continuous MaxRL (2340.40 ± 914.41). In addition, its variance across seeds is substantially lower than the other two

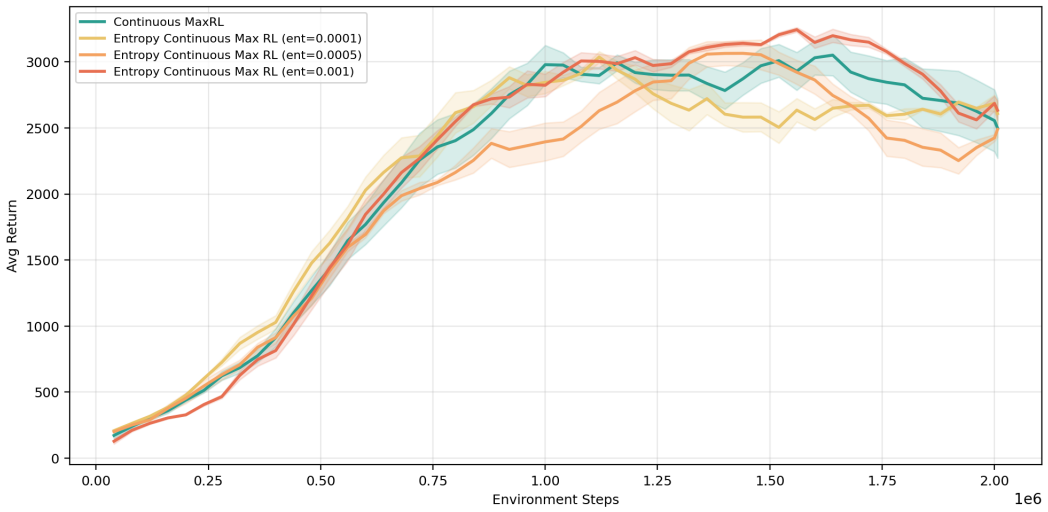

Figure 6: Effect of different entropy coefficients on Hopper-v5 with 2000k training steps.

baselines, indicating that entropy regularization improves not only the final performance but also the robustness of training. Therefore, for Hopper-v5, adding a suitable entropy term to Continuous MaxRL leads to a clear and consistent gain.

Figure 5 presents a slightly different pattern on Walker2d-v5. PPO achieves the highest mean return, reaching 3330.53 ± 1091.93, while Entropy Continuous MaxRL with entropy coefficient 0.0005 obtains a very similar return of 3276.50 ± 108.26. Continuous MaxRL without entropy regularization reaches 3256.54 ± 575.09, which is also competitive but slightly lower. Although PPO has the highest average return, its variance is much larger than the entropy-regularized variant. This suggests that, on Walker2d-v5, Entropy Continuous MaxRL does not clearly surpass PPO in absolute score, but it provides a considerably more stable optimization process across different random seeds.

Overall, the two environments reveal complementary conclusions. On Hopper-v5, entropy-regularized Continuous MaxRL is clearly the strongest method in both return and stability. On Walker2d-v5, PPO remains marginally better in mean return, but Entropy Continuous MaxRL offers a better robustness-performance tradeoff due to its much smaller variance. These results suggest that entropy regularization is particularly effective for improving the consistency of policy optimization in continuous-control tasks.

To further understand the role of entropy regularization, we compare Continuous MaxRL with its entropy-regularized variants under three different entropy coefficients: 0.0001, 0.0005, and 0.001. The corresponding return curves on Hopper-v5 and Walker2d-v5 are shown in Figure 6 and Figure 7.

On Hopper-v5, the entropy coefficient has a clear impact on both final return and training stability. Without entropy regularization, Continuous MaxRL achieves an average return of 2340.40 ± 914.41. Adding entropy with coefficient 0.0005 improves the result substantially to 2889.49 ± 391.30, which is the best performance among all tested entropy settings. When the coefficient is reduced to 0.0001, performance drops to 1991.17 ± 917.37, indicating that the exploration bonus is too weak to provide a meaningful benefit. Increasing the coefficient to 0.001 still gives relatively strong performance, with return 2792.32 ± 1050.15, but the variance becomes much larger. Therefore, on Hopper-v5, an intermediate entropy coefficient yields the best balance between return and stability.

A similar trend appears on Walker2d-v5. Continuous MaxRL without entropy achieves 3256.54 ± 575.09, while entropy regularization with coefficient 0.0005 slightly improves the mean return to 3276.50 ± 108.26 and greatly reduces the variance across seeds. In contrast, coefficient 0.0001 leads to a much lower return of 2043.21 ± 397.70, and coefficient 0.001 reaches 3083.45 ± 516.96, which is better than 0.0001 but still worse than 0.0005. This suggests that Walker2d-v5 is also sensitive to the entropy parameter, and that the benefit of entropy regularization depends strongly on selecting an appropriate magnitude.

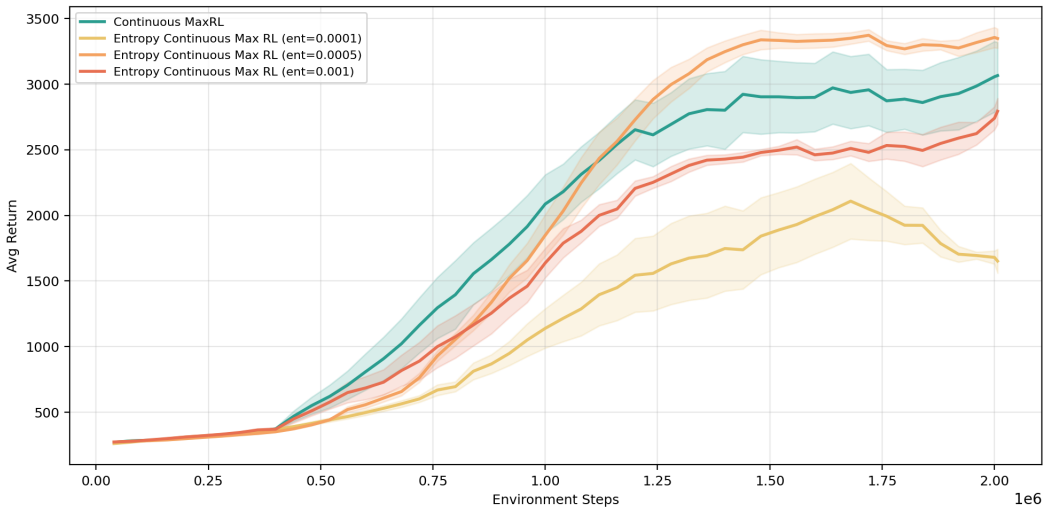

Figure 7: Effect of different entropy coefficients on Walker2d-v5 with 2000k training steps.

Overall, the entropy sweep experiments show that entropy regularization is beneficial, but only when the coefficient is properly tuned. In both Hopper-v5 and Walker2d-v5, the intermediate setting 0.0005 gives the strongest overall result, while values that are too small or too large lead to weaker performance or higher instability. These results indicate that entropy regularization mainly helps by improving exploration and stabilizing optimization, but its effect is highly parameter-sensitive.

6 Analysis on the GSM8K Task

6.1 GSM8K Language-Model Reasoning Experiment (distilgpt2)

To further evaluate whether the proposed Continuous MaxRL framework can be applied beyond standard control environments, we conduct an additional experiment on the GSM8K mathematical reasoning dataset. This experiment is intended as a small-scale language-model reasoning test rather than a fully optimized large-language-model training run. We use `distilgpt2` as the base model due to computational constraints and train it using policy-gradient-style updates over sampled generations.

Experimental Setup. For each GSM8K question, the model is prompted to generate a numerical answer. We sample multiple responses from the current policy and compute rewards based on the generated answer. Since exact correctness is extremely sparse for a small base model, we use a partial reward design:

$$r_i = \begin{cases} 0.05, & \text{if the model produces a numerical answer,} \\ 0, & \text{otherwise,} \end{cases}$$

with an additional reward of 1.0 if the extracted numerical answer exactly matches the ground-truth answer. Thus, producing a valid numerical output receives a small reward, while producing the correct answer receives a much larger reward.

For each training iteration, we sample a batch of GSM8K prompts and generate multiple responses per prompt. In our implementation, we use batch size 8, 8 samples per prompt, and train for 200 iterations. The optimizer is AdamW with learning rate 10^{-5}. For entropy-regularized Continuous MaxRL, we use an entropy coefficient $\lambda = 5 \times 10^{-5}$, as reported in Section 4.4, where this value achieved the best performance in the LunarLander environment. All methods are trained from the same base model initialization for fair comparison.

We compare four methods:

- REINFORCE,
- Binary MaxRL,

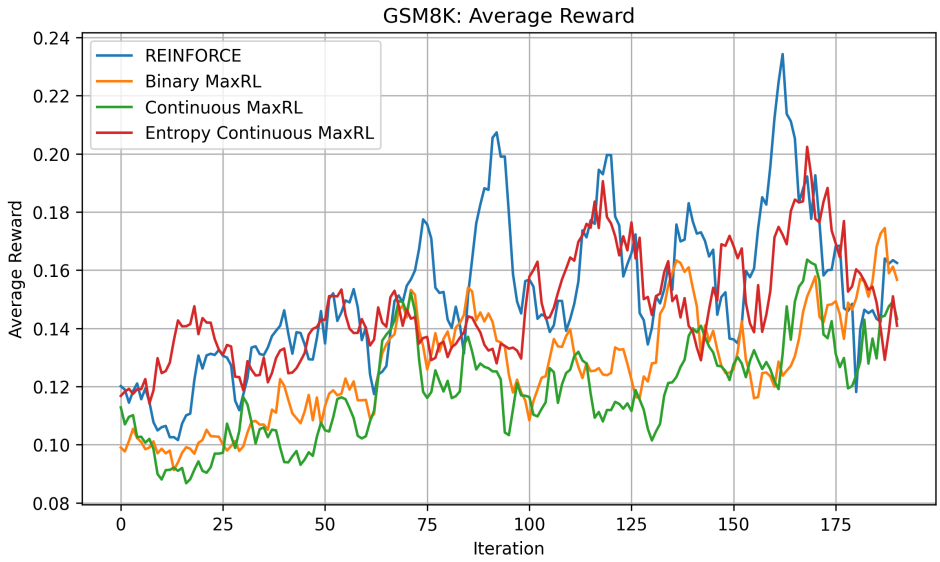

Figure 8: Average reward on GSM8K for REINFORCE, Binary MaxRL, Continuous MaxRL, and Entropy-regularized Continuous MaxRL.

- Continuous MaxRL,
- Entropy-regularized Continuous MaxRL.

Implementation Details. For each generated response, we recompute the log-likelihood of the generated tokens under the current model. The policy-gradient update is applied at the sequence level. Specifically, for each sampled response τ_i, we compute

$$\log \pi_\theta(\tau_i) = \sum_t \log \pi_\theta(y_{i,t} \mid y_{i,<t}, x_i),$$

where x_i is the GSM8K prompt and $y_{i,t}$ are generated tokens. The loss is then computed as

$$\mathcal{L}(\theta) = -\sum_i w_i \log \pi_\theta(\tau_i) - \lambda H(\pi_\theta),$$

where w_i is determined by the corresponding method. For REINFORCE, w_i is proportional to the reward. For Binary MaxRL, w_i is based only on whether the response is correct. For Continuous MaxRL, the reward values are shifted to be nonnegative and normalized across the rollout batch:

$$w_i = \frac{\widetilde{R}_i}{\sum_j \widetilde{R}_j + \epsilon}.$$

6.2 Results for GSM8K

Figure 8 shows the average reward over training. Overall, all methods achieve modest performance, reflecting the difficulty of GSM8K for a small base model such as `distilgpt2`. REINFORCE obtains the highest reward spikes, with a maximum average reward of approximately 0.446, but its learning dynamics are unstable. Binary MaxRL maintains more gradual behavior but achieves lower peak performance, with maximum average reward around 0.277. Continuous MaxRL without entropy achieves similar but slightly weaker performance, with maximum average reward around 0.279. Entropy-regularized Continuous MaxRL improves over the non-regularized Continuous MaxRL, achieving a higher maximum average reward of approximately 0.396.

Figure 9 reports the success rate, defined as the fraction of generated responses whose extracted numerical answer exactly matches the ground-truth answer. REINFORCE reaches the highest success spike of 0.25, but this improvement is not sustained. Entropy-regularized Continuous MaxRL

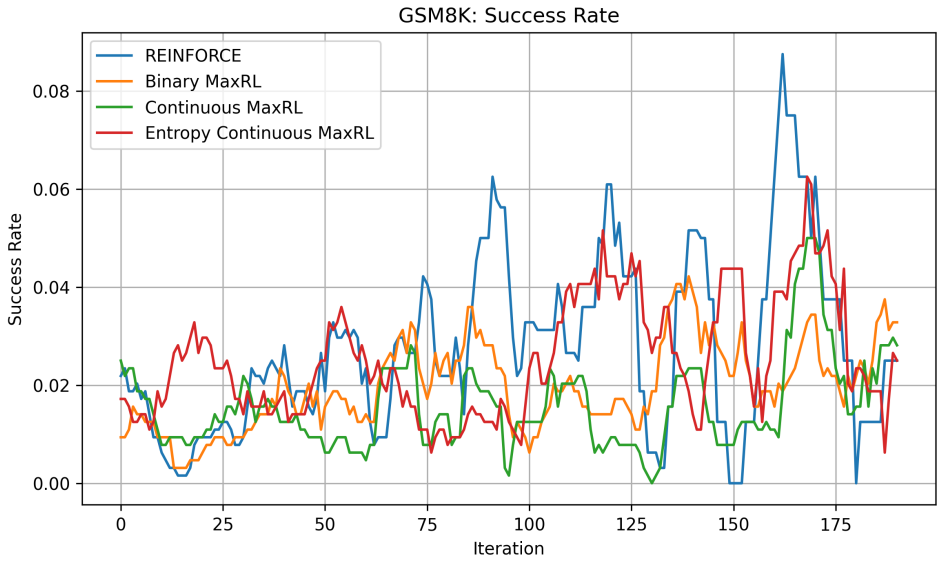

Figure 9: Success rate on GSM8K. Success is defined as exact numerical answer match.

reaches a maximum success rate of 0.234, which is substantially higher than Binary MaxRL and non-regularized Continuous MaxRL. Binary MaxRL reaches a maximum success rate of 0.094, while Continuous MaxRL reaches 0.125. These results suggest that entropy regularization helps Continuous MaxRL discover successful generations more frequently than the non-regularized version.

Figure 10 shows the policy entropy during training. REINFORCE exhibits rapid entropy collapse, decreasing from high initial entropy to nearly zero by the end of training. This suggests that REINFORCE quickly concentrates probability mass on a narrow set of generations, which may explain its unstable reward and success spikes. Binary MaxRL maintains entropy for longer, but its improvement in reward and success remains limited. Continuous MaxRL also reduces entropy over time, but more gradually than REINFORCE. Entropy-regularized Continuous MaxRL maintains slightly more exploration than REINFORCE in the middle of training and achieves better performance than the non-regularized Continuous MaxRL.

6.2.1 Discussion

The GSM8K experiment provides a useful stress test for applying MaxRL-style objectives to language-model reasoning. Unlike LunarLander, where rewards are naturally dense and directly tied to task performance, GSM8K requires the model to generate a correct mathematical answer. For a small model such as `distilgpt2`, exact correctness remains rare, and the reward signal is therefore weak. As a result, Continuous MaxRL alone does not dominate all baselines. However, the entropy-regularized variant improves over the non-regularized Continuous MaxRL in both average reward and success rate, indicating that maintaining exploration is important in reasoning tasks.

These results suggest that continuous reward normalization is useful, but not sufficient by itself for difficult language-model reasoning tasks. Effective application to GSM8K likely requires stronger base models, better reward shaping, or verifier-based continuous rewards. Nevertheless, the experiment demonstrates that the proposed framework can be applied to language-model generation and highlights the importance of exploration control when optimizing likelihood-based reinforcement learning objectives for reasoning.

7 Conclusion

We proposed a continuous-reward extension of Maximum Likelihood Reinforcement Learning by replacing success-count normalization with reward-mass normalization. This modification allows MaxRL-style policy-gradient updates to use dense return information instead of relying only on

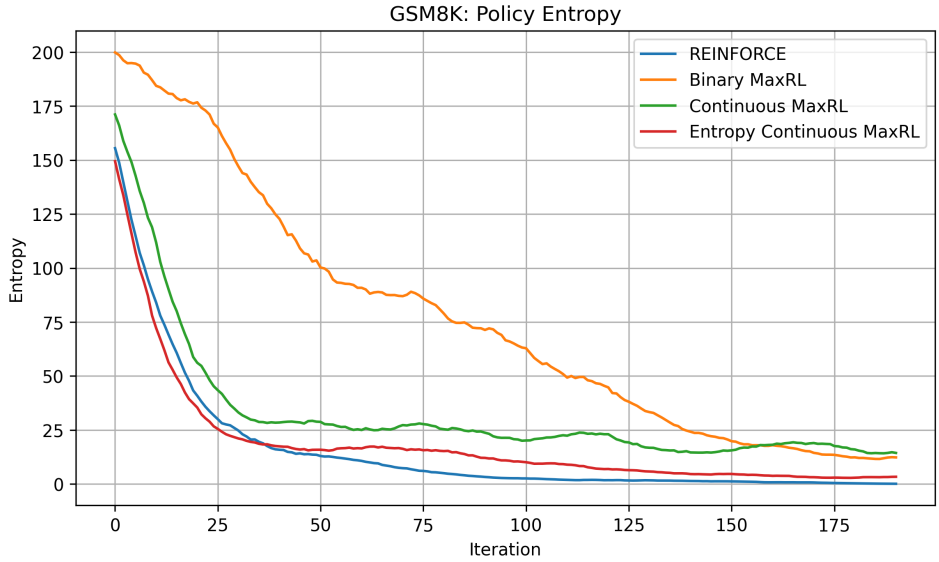

Figure 10: Policy entropy on GSM8K. Entropy collapse indicates that the model becomes increasingly deterministic during training.

binary success indicators. We also introduced an entropy-regularized variant to improve exploration and avoid premature policy collapse. Experiments on LunarLander-v3, Hopper-v5, Walker2d-v5, and Acrobot-v1 show that Continuous MaxRL improves learning stability compared with Binary MaxRL, and that properly tuned entropy regularization can further improve performance and robustness.

The GSM8K experiments suggest that the same ideas can be applied to language-model reasoning, although stronger models and better reward signals are needed for more reliable performance. Overall, our findings indicate that likelihood-inspired reward normalization can be generalized beyond binary rewards and may provide a useful direction for reinforcement learning in both control and reasoning domains. Future work includes combining Continuous MaxRL with more stable RL algorithms, designing adaptive entropy schedules, and applying verifier-based continuous rewards to stronger language models.

References

Damek Davis and Benjamin Recht. What is the objective of reasoning with reinforcement learning? 2025.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. Pmlr, 2018.

John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*, volume 1.

Fahim Tajwar, Xiang Zeng, Shunyu Zhou, Dawn Song, Sanjeev Arora, Nan Jiang, Jeff Schneider, Ruslan Salakhutdinov, Jiashi Feng, and Andrea Zanette. Maximum likelihood reinforcement learning. 2026.

Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3):229–256, 1992.

Wei Xiong, Chenlu Ye, Baohao Liao, Hanze Dong, Xinxing Xu, Christof Monz, Jiang Bian, Nan Jiang, and Tong Zhang. Reinforce-ada: An adaptive sampling framework under non-linear rl objectives. *arXiv preprint arXiv:2510.04996*, 2025.

Xiang Zeng, Yuanzhi Li, Alfredo Garcia, and Mingyi Hong. Maximum-likelihood inverse reinforcement learning with finite-time guarantees. In *NeurIPS*, 2022.

Xiang Zeng, Yuanzhi Li, Alfredo Garcia, and Mingyi Hong. When demonstrations meet generative world models: A maximum likelihood framework for offline inverse reinforcement learning. In *NeurIPS*, 2023.

Brian D Ziebart, J Andrew Bagnell, Anind K Dey, et al. Maximum entropy inverse reinforcement learning. 2008.

A Theoretical Insight

A.1 Sensitivity to Entropy Regularization

We first analyze how the entropy coefficient affects the update direction. Consider the entropy-regularized Continuous-Reward MaxRL objective

$$J_\lambda(\theta) = J_{\mathrm{CR-MaxRL}}(\theta) + \lambda H(\pi_\theta),$$

where $J_{\mathrm{CR-MaxRL}}(\theta)$ denotes the reward-normalized MaxRL objective, $H(\pi_\theta)$ denotes the policy entropy, and $\lambda \geq 0$ is the entropy coefficient.

Proposition 1 (Linear sensitivity to entropy regularization)**.** *For any two entropy coefficients $\lambda_1, \lambda_2 \geq 0$, suppose that the entropy gradient is bounded, i.e.,*

$$\|\nabla H(\pi_\theta)\| \leq G_H.$$

Then

$$\|\nabla J_{\lambda_1}(\theta) - \nabla J_{\lambda_2}(\theta)\| \leq |\lambda_1 - \lambda_2| G_H.$$

Proof. By definition,

$$\nabla J_\lambda(\theta) = \nabla J_{\mathrm{CR-MaxRL}}(\theta) + \lambda \nabla H(\pi_\theta).$$

Therefore,

$$\nabla J_{\lambda_1}(\theta) - \nabla J_{\lambda_2}(\theta) = (\lambda_1 - \lambda_2)\nabla H(\pi_\theta).$$

Taking norms on both sides gives

$$\|\nabla J_{\lambda_1}(\theta) - \nabla J_{\lambda_2}(\theta)\| = |\lambda_1 - \lambda_2| \|\nabla H(\pi_\theta)\|.$$

Using the bounded-gradient assumption $\|\nabla H(\pi_\theta)\| \leq G_H$, we obtain

$$\|\nabla J_{\lambda_1}(\theta) - \nabla J_{\lambda_2}(\theta)\| \leq |\lambda_1 - \lambda_2| G_H.$$

□

This proposition shows that the update direction changes at most linearly with respect to the entropy coefficient. Hence, a small value of λ only mildly perturbs the reward-normalized MaxRL update, whereas a large value of λ can substantially alter the learning direction.

Indeed, the entropy-regularized update takes the form

$$\nabla J_\lambda(\theta) = \nabla J_{\mathrm{CR-MaxRL}}(\theta) + \lambda \nabla H(\pi_\theta).$$

A sufficient condition for the entropy term not to dominate the reward-normalized update is

$$\lambda \|\nabla H(\pi_\theta)\| < \|\nabla J_{\mathrm{CR-MaxRL}}(\theta)\|.$$

Equivalently, whenever $\|\nabla H(\pi_\theta)\| > 0$, it suffices to choose

$$\lambda < \frac{\|\nabla J_{\mathrm{CR-MaxRL}}(\theta)\|}{\|\nabla H(\pi_\theta)\|}.$$

This condition explains the empirical role of entropy regularization: moderate entropy regularization can improve exploration and prevent premature policy collapse, while overly large entropy coefficients may disrupt reward-driven learning.

A.2 Convergence Interpretation

Entropy-regularized Continuous-Reward MaxRL can be interpreted as stochastic gradient ascent on a regularized reward-weighted likelihood surrogate. Given a batch of trajectories

$$\mathcal{B}_t = \{\tau_{t,i}\}_{i=1}^{N}$$

sampled from the current policy π_{θ_t}, let $R_{t,i} = R(\tau_{t,i})$ be the episodic return. Since returns may be negative, we use the shifted nonnegative reward mass

$$\widetilde{R}_{t,i} = \max\left(R_{t,i} - \min_j R_{t,j}, 0\right),$$

and define the normalized weight

$$w_{t,i} = \frac{\widetilde{R}_{t,i}}{\sum_{j=1}^{N} \widetilde{R}_{t,j} + \epsilon}.$$

The stochastic update direction used by Entropy-Regularized Continuous-Reward MaxRL is

$$g_t = \sum_{i=1}^{N} w_{t,i} \nabla_\theta \log \pi_{\theta_t}(\tau_{t,i}) + \lambda \nabla_\theta H(\pi_{\theta_t}).$$

Equivalently, this is the gradient of the batch-level surrogate

$$\widehat{J}_\lambda(\theta; \mathcal{B}_t) = \sum_{i=1}^{N} w_{t,i} \log \pi_\theta(\tau_{t,i}) + \lambda H(\pi_\theta),$$

where the weights $w_{t,i}$ are computed from the sampled batch and treated as fixed coefficients during the policy update. The use of batch-normalized weights introduces a bias in the gradient estimator, since the weights depend on the sampled batch rather than the underlying data distribution.

Because neural network policies make the objective nonconvex, we do not claim convergence to a global optimum. Instead, we use the standard stationarity perspective from nonconvex stochastic optimization. Suppose that J_λ is L-smooth, namely

$$\|\nabla J_\lambda(\theta_1) - \nabla J_\lambda(\theta_2)\| \le L\|\theta_1 - \theta_2\|$$

for all θ_1, θ_2. Assume that the stochastic gradient estimator g_t has bounded variance and is either unbiased or has bounded bias:

$$\|\mathbb{E}[g_t \mid \theta_t] - \nabla J_\lambda(\theta_t)\| \le b,$$

and

$$\mathbb{E}\left[\|g_t - \mathbb{E}[g_t \mid \theta_t]\|^2 \mid \theta_t\right] \le \sigma^2.$$

The case $b = 0$ corresponds to an unbiased stochastic gradient estimator, while $b > 0$ captures the possible bias introduced by reward normalization and finite-batch weighting.

For simplicity, define $d_t = \nabla J_\lambda(\theta_t)$. Assume that the stochastic gradient estimator has bounded bias

$$\mathbb{E}[g_t \mid \theta_t] = d_t + \delta_t, \qquad \|\delta_t\| \le b,$$

and bounded variance

$$\mathbb{E}\left[\|g_t - \mathbb{E}[g_t \mid \theta_t]\|^2 \mid \theta_t\right] \le \sigma^2.$$

By L-smoothness and the gradient ascent update

$$\theta_{t+1} = \theta_t + \eta g_t,$$

we have

$$J_\lambda(\theta_{t+1}) \ge J_\lambda(\theta_t) + \eta \langle d_t, g_t \rangle - \frac{L\eta^2}{2}\|g_t\|^2.$$

Taking conditional expectation with respect to θ_t, we obtain

$$\mathbb{E}\left[J_\lambda(\theta_{t+1}) \mid \theta_t\right] \ge J_\lambda(\theta_t) + \eta \langle d_t, \mathbb{E}[g_t \mid \theta_t] \rangle - \frac{L\eta^2}{2}\mathbb{E}\left[\|g_t\|^2 \mid \theta_t\right].$$

For the first-order term, using

$$\mathbb{E}[g_t \mid \theta_t] = d_t + \delta_t,$$

we have

$$\langle d_t, \mathbb{E}[g_t \mid \theta_t] \rangle = \langle d_t, d_t + \delta_t \rangle = \|d_t\|^2 + \langle d_t, \delta_t \rangle.$$

Since

$$\langle d_t, \delta_t \rangle \geq -\|d_t\|\|\delta_t\| \geq -b\|d_t\|,$$

and by Young's inequality,

$$b\|d_t\| \leq \frac{1}{4}\|d_t\|^2 + b^2,$$

we obtain

$$\langle d_t, \mathbb{E}[g_t \mid \theta_t] \rangle \geq \frac{3}{4}\|d_t\|^2 - b^2.$$

For the second-order term, by the variance decomposition,

$$\mathbb{E}\left[\|g_t\|^2 \mid \theta_t\right] = \|\mathbb{E}[g_t \mid \theta_t]\|^2 + \mathbb{E}\left[\|g_t - \mathbb{E}[g_t \mid \theta_t]\|^2 \mid \theta_t\right].$$

Using the bounded-variance assumption,

$$\mathbb{E}\left[\|g_t\|^2 \mid \theta_t\right] \leq \|d_t + \delta_t\|^2 + \sigma^2.$$

Furthermore,

$$\|d_t + \delta_t\|^2 \leq 2\|d_t\|^2 + 2\|\delta_t\|^2 \leq 2\|d_t\|^2 + 2b^2.$$

Therefore,

$$\mathbb{E}\left[\|g_t\|^2 \mid \theta_t\right] \leq 2\|d_t\|^2 + 2b^2 + \sigma^2.$$

Substituting these two bounds into the one-step improvement inequality gives

$$\mathbb{E}\left[J_\lambda(\theta_{t+1}) \mid \theta_t\right] \geq J_\lambda(\theta_t) + \eta\left(\frac{3}{4}\|d_t\|^2 - b^2\right) - \frac{L\eta^2}{2}\left(2\|d_t\|^2 + 2b^2 + \sigma^2\right).$$

Rearranging terms yields

$$\mathbb{E}\left[J_\lambda(\theta_{t+1}) \mid \theta_t\right] \geq J_\lambda(\theta_t) + \left(\frac{3\eta}{4} - L\eta^2\right)\|d_t\|^2 - \eta b^2 - L\eta^2 b^2 - \frac{L\eta^2\sigma^2}{2}.$$

If the stepsize satisfies

$$\eta \leq \frac{1}{4L},$$

then

$$\frac{3\eta}{4} - L\eta^2 \geq \frac{\eta}{2}.$$

Thus,

$$\mathbb{E}\left[J_\lambda(\theta_{t+1}) \mid \theta_t\right] \geq J_\lambda(\theta_t) + \frac{\eta}{2}\|d_t\|^2 - \eta b^2 - L\eta^2 b^2 - \frac{L\eta^2\sigma^2}{2}.$$

Equivalently,

$$\frac{\eta}{2}\|d_t\|^2 \leq \mathbb{E}\left[J_\lambda(\theta_{t+1}) \mid \theta_t\right] - J_\lambda(\theta_t) + \eta b^2 + L\eta^2 b^2 + \frac{L\eta^2\sigma^2}{2}.$$

Taking total expectation and summing over $t = 0, \ldots, T-1$, we obtain

$$\frac{\eta}{2}\sum_{t=0}^{T-1}\mathbb{E}\left[\|d_t\|^2\right] \leq \mathbb{E}[J_\lambda(\theta_T)] - J_\lambda(\theta_0) + T\eta b^2 + TL\eta^2 b^2 + \frac{TL\eta^2\sigma^2}{2}.$$

Assume that the objective is upper bounded by $J_\lambda^\star$, i.e.,

$$J_\lambda(\theta) \leq J_\lambda^\star \quad \text{for all } \theta.$$

Then

$$\frac{\eta}{2}\sum_{t=0}^{T-1}\mathbb{E}\left[\|d_t\|^2\right] \leq J_\lambda^\star - J_\lambda(\theta_0) + T\eta b^2 + TL\eta^2 b^2 + \frac{TL\eta^2\sigma^2}{2}.$$

Dividing both sides by $T\eta/2$, we get

$$\frac{1}{T}\sum_{t=0}^{T-1}\mathbb{E}\left[\|\nabla J_\lambda(\theta_t)\|^2\right] \le \frac{2\left(J_\lambda^\star - J_\lambda(\theta_0)\right)}{T\eta} + 2b^2 + 2L\eta b^2 + L\eta\sigma^2.$$

Choosing $\eta = O(T^{-1/2})$ gives

$$\frac{1}{T}\sum_{t=0}^{T-1}\mathbb{E}\left[\|\nabla J_\lambda(\theta_t)\|^2\right] = O(T^{-1/2}) + O(b^2).$$

In the unbiased case $b = 0$, this reduces to

$$\frac{1}{T}\sum_{t=0}^{T-1}\mathbb{E}\left[\|\nabla J_\lambda(\theta_t)\|^2\right] = O(T^{-1/2}).$$

Therefore, under standard smoothness, bounded-variance, and bounded-bias assumptions, Entropy-Regularized Continuous-Reward MaxRL converges to a neighborhood of a stationary point of the entropy-regularized objective. When the stochastic gradient estimator is unbiased, the standard $O(T^{-1/2})$ stationarity rate is recovered.

This result means that the average squared gradient norm decreases with the number of iterations, up to a possible bias-dependent neighborhood. Therefore, Entropy-Regularized Continuous-Reward MaxRL approaches a stationary point, or a neighborhood of a stationary point, of the entropy-regularized surrogate objective. This does not imply global optimality because the policy class is nonconvex, but it shows that the proposed update is well-posed as a stochastic optimization procedure.

This interpretation also explains the empirical sensitivity to the entropy coefficient. The parameter λ changes the objective being optimized by adding the entropy-gradient term $\lambda\nabla H(\pi_\theta)$. A moderate value of λ encourages exploration and improves stability, while an overly large value may cause the entropy term to dominate the reward-normalized MaxRL gradient and interfere with reward-driven learning.

B Additional Experiment

B.1 Acrobot-v1 Experiment

To evaluate the proposed Continuous-Reward MaxRL method in a standard reinforcement learning environment, we conduct experiments on `Acrobot-v1` from Gymnasium. Acrobot is a classic control environment with a two-link underactuated system. The goal is to swing the end effector above a target height using a discrete torque action. This environment is useful for our setting because it provides a dense but negative reward signal: the agent receives approximately -1 reward at each timestep until the goal is reached. Therefore, an episode return of -500 indicates that the agent failed to reach the goal within the maximum episode length of 500 steps, while a higher return such as -80 indicates that the agent reached the goal much earlier.

Unlike LunarLander, where successful performance is often associated with a positive return threshold, Acrobot returns are negative. Therefore, we define success using a negative threshold:

$$R(\tau) \ge -100.$$

Under this definition, a trajectory is counted as successful if the agent reaches the goal in roughly fewer than 100 steps. This threshold allows us to distinguish between partial improvement and successful task completion.

Experimental Setup. We compare REINFORCE, Binary MaxRL, Continuous MaxRL, and entropy-regularized Continuous MaxRL. All methods use the same policy architecture and optimization settings. The policy is a two-layer MLP with hidden size 128 and Tanh activations. Since Acrobot has a discrete action space, the policy outputs categorical action probabilities. We train each method for 800 iterations, with a maximum episode length of 500 steps. The optimizer is Adam with learning rate 10^{-3}, and the discount factor is $\gamma = 0.99$.

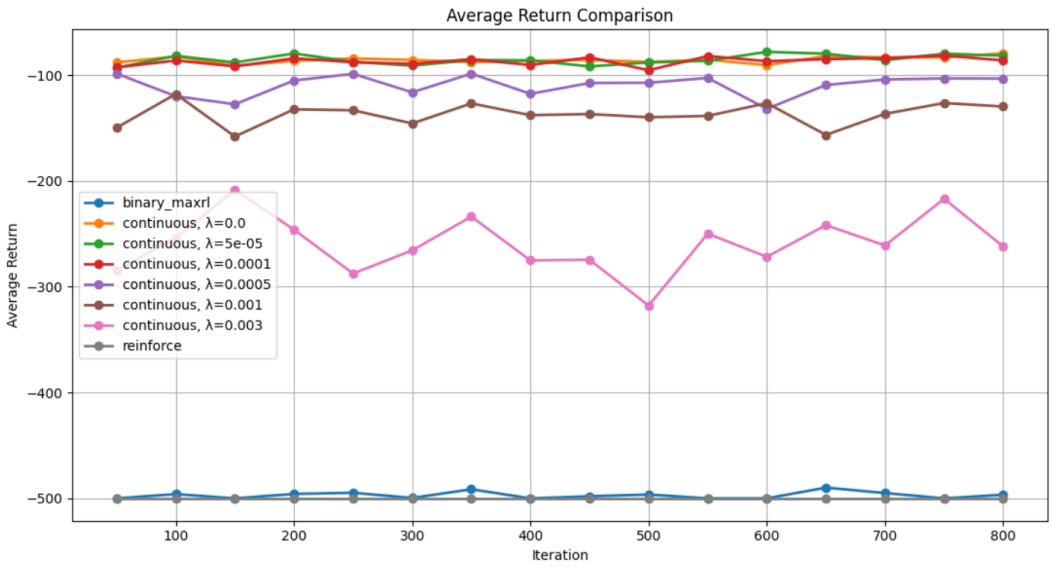

Figure 11: Average return comparison on Acrobot-v1. Continuous MaxRL significantly outperforms Reinforce and Binary MaxRL.

For entropy-regularized Continuous MaxRL, we sweep over the entropy coefficient

$$\lambda \in \{0,\ 5 \times 10^{-5},\ 10^{-4},\ 5 \times 10^{-4},\ 10^{-3},\ 3 \times 10^{-3}\}.$$

The loss function for entropy-regularized Continuous MaxRL is

$$L(\theta) = -\sum_i w_i \log \pi_\theta(\tau_i) - \lambda H(\pi_\theta),$$

where the Continuous MaxRL weights are computed using shifted nonnegative returns:

$$\tilde{R}_i = \max(R_i - \min_j R_j, 0), \qquad w_i = \frac{\tilde{R}_i}{\sum_j \tilde{R}_j + \epsilon}.$$

This weighting scheme emphasizes relatively better trajectories in the batch, even when no trajectory reaches the success threshold.

Results: Average Return. Figure 11 shows the average return comparison across methods. REINFORCE remains near -500 throughout training, indicating that the policy fails to solve the task and typically reaches the maximum episode length. Binary MaxRL also stays close to -500 because successful trajectories are rare, so the binary success-based update often provides little or no useful gradient signal.

In contrast, Continuous MaxRL achieves substantially higher returns. The non-regularized setting $\lambda = 0$ and the small-entropy setting $\lambda = 5 \times 10^{-5}$ reach average returns around -80 to -90, showing that reward-mass normalization provides a useful learning signal even when binary success feedback is sparse. This result supports our hypothesis that continuous reward normalization can better exploit dense return information than binary success normalization.

However, larger entropy coefficients degrade performance. When $\lambda = 5 \times 10^{-4}$, the return becomes noticeably worse and less stable. When $\lambda = 10^{-3}$ or $\lambda = 3 \times 10^{-3}$, performance drops substantially. This indicates that excessive entropy regularization prevents the policy from becoming sufficiently goal-directed.

Results: Success Rate. Figure 12 shows the success rate under the threshold $R(\tau) \geq -100$. REINFORCE and Binary MaxRL achieve zero success throughout training. Continuous MaxRL, however, reaches high success rates for small entropy coefficients. In particular, $\lambda = 0$, $\lambda = 5 \times 10^{-5}$,

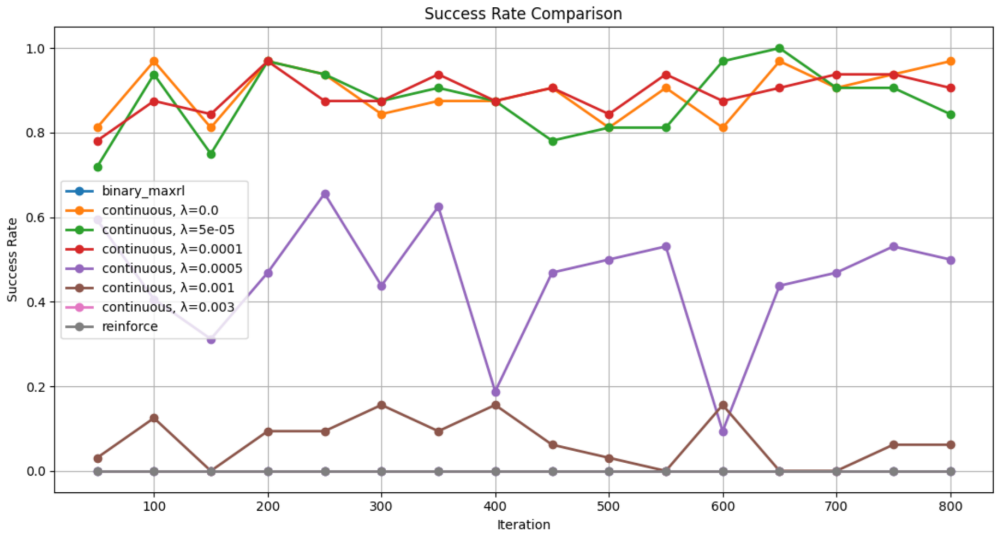

Figure 12: Success rate comparison on Acrobot-v1 using the threshold $R(\tau) \geq -100$

and $\lambda = 10^{-4}$ frequently achieve success rates above 0.8, with some iterations approaching or reaching 1.0.

This demonstrates that the improvement in average return corresponds to meaningful task completion rather than only small changes in reward. The success-rate curve also confirms the sensitivity of entropy regularization: small entropy coefficients preserve strong success performance, while larger entropy coefficients reduce the probability of reaching the goal.

Results: Policy Entropy. Figure 13 reports the policy entropy during training. The entropy curves help explain the performance differences among entropy coefficients. For small values of λ, policy entropy decreases over training, suggesting that the policy gradually becomes more deterministic as it discovers useful behaviors. This is desirable because once the agent finds effective swing-up actions, exploitation becomes important.

For larger entropy coefficients, entropy remains much higher. Although this encourages exploration, it also prevents the policy from consistently exploiting high-return trajectories. As a result, larger values such as $\lambda = 10^{-3}$ and $\lambda = 3 \times 10^{-3}$ lead to worse return and lower success rate. Therefore, entropy regularization must be carefully tuned: too little exploration may cause premature collapse, but too much exploration prevents stable improvement.

Discussion. The Acrobot results show that Continuous MaxRL is much more effective than Binary MaxRL in environments where exact success is initially rare but dense reward information is available. Binary MaxRL discards useful information from partially successful trajectories, while Continuous MaxRL uses relative return values to assign gradient weights. This allows the policy to improve even before consistent successes are observed.

B.2 GSM8K Pass@k Evaluation with Qwen2.5-Math-7B-Instruct

To further study the connection between rollout sampling and reasoning performance, we conduct an additional GSM8K evaluation using Qwen2.5-Math-7B-Instruct. Unlike the previous policy-gradient experiments, this experiment does not update model parameters. Instead, it evaluates whether sampling multiple reasoning trajectories improves the probability of obtaining a correct answer. This provides a language-model reasoning analogue of the MaxRL idea, where a policy samples multiple candidate trajectories and evaluation checks whether any of them achieves high reward.

Experimental Setup We use Qwen2.5-Math-7B-Instruct as the base model because it is designed for mathematical reasoning and can generate step-by-step solutions. The model is evaluated on a

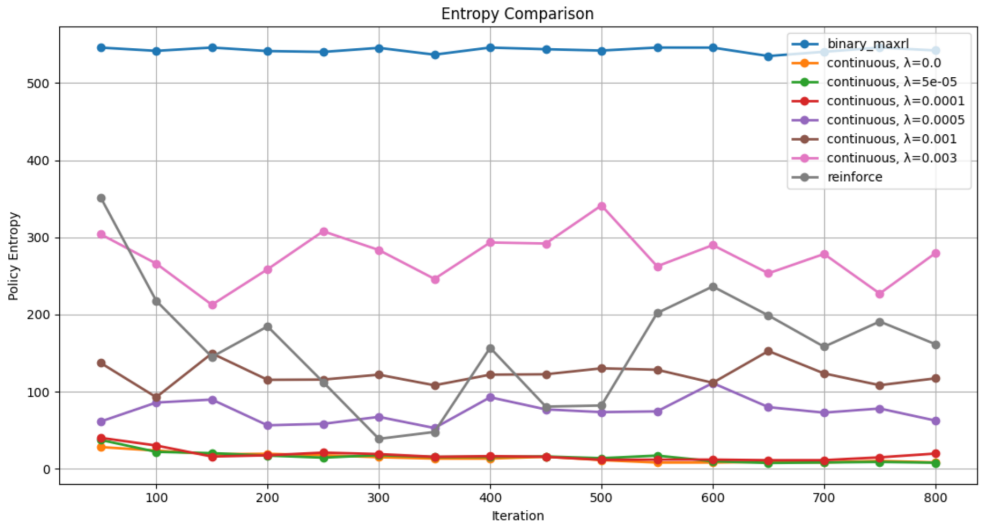

Figure 13: Policy entropy comparison on Acrobot-v1. Larger entropy coefficients maintain higher entropy but reduce task performance.

subset of the GSM8K test set. For each question, we generate up to eight sampled responses using nucleus sampling. The decoding parameters are:

$$k \in \{1, 4, 8\}, \qquad \text{max\_new\_tokens} = 512, \qquad \text{top\_p} = 0.95.$$

For each GSM8K problem, the model is prompted to solve the question step by step and output a final numerical answer. Each generated solution is treated as one reasoning rollout. A rollout is considered successful if the extracted answer exactly matches the ground-truth answer. We evaluate three pass@k metrics:

$$\text{pass@}k = \mathbb{I}\left[\max_{i \in \{1,\ldots,k\}} r_i = 1\right].$$

where $r_i = 1$ if the i-th generated solution is correct and $r_i = 0$ otherwise. In our experiments, we report pass@1, pass@4, and pass@8. pass@1 measures single-sample accuracy, while pass@4 and pass@8 measure whether at least one correct answer appears among multiple sampled reasoning rollouts.

This setup is closely related to MaxRL, but it should be interpreted as an evaluation experiment rather than a training algorithm. In MaxRL, multiple trajectories are sampled and high-reward trajectories influence the policy update. In our GSM8K pass@k experiment, multiple reasoning trajectories are sampled but the model weights are not updated. Therefore, pass@k measures the potential benefit of additional rollout compute at inference time.

In addition to pass@k, we also compute majority-vote accuracy. For majority@k, the model generates k solutions, the final numerical answer is extracted from each response, and the most frequent answer is selected. This is a more realistic non-oracle aggregation method because it does not use the ground-truth answer to select the best sample.

Implementation Details The GSM8K pass@k experiment is implemented as an inference-only evaluation pipeline. We first construct a chat-style prompt for each GSM8K problem using the Qwen2.5-Math-7B-Instruct tokenizer. The prompt asks the model to solve the problem step by step and end with the format `Final answer: <number>`. For each question, we generate up to $k = 8$ candidate solutions using nucleus sampling with temperature 0.7, top-p $= 0.95$, and maximum generation length of 512 tokens.

After generation, we apply an answer-extraction function to each response. This function first searches for the explicit pattern `Final answer:`; if this pattern is not present, it falls back to extracting the last numerical value in the generated response. The extracted number is normalized by removing commas, dollar signs, and percent signs, and then converted to a floating-point value. The ground-truth GSM8K

answer is extracted from the official answer field after the delimiter. A generated solution is marked correct if its extracted numerical answer matches the normalized ground-truth answer within a small numerical tolerance.

For each problem, we generate eight sampled solutions once and reuse the first 1, 4, and 8 samples to compute pass@1, pass@4, and pass@8. pass@k is counted as correct if any of the first k generated answers matches the ground truth. We also compute majority@k by selecting the most frequent extracted numerical answer among the first k samples. Unlike pass@k, majority voting does not use the ground-truth answer for selection, so it serves as a non-oracle aggregation baseline.

B.2.1 Result

Figure 14 shows increasing the number of sampled reasoning rollouts improves pass@k, indicating that additional rollout compute can reveal correct solutions that are missed by a single generation. However, majority-vote accuracy improves only modestly, showing that the correct answer is not always the most frequent sample. Therefore, pass@k should be viewed as an oracle upper bound, while majority@k is a more realistic non-oracle aggregation baseline.

This experiment is closely related to the MaxRL motivation, but it is not RL training. In MaxRL, the policy samples multiple trajectories and high-reward trajectories are used to guide the policy update. In the GSM8K pass@k experiment, the language model samples multiple reasoning trajectories, but the model weights are not updated. Therefore, this experiment should be interpreted as a MaxRL-inspired inference-time rollout evaluation. It tests whether additional sampling compute improves the probability of finding at least one correct solution.

We use this inference-only setup because Qwen2.5-Math-7B is a large pretrained model, and full RL fine-tuning would require substantially more computation, memory, and stabilization techniques. In addition, our goal in this experiment is to evaluate the benefit of multiple reasoning rollouts rather than to train a new model. The Acrobot-v1 experiment provides the actual RL training evaluation, while the GSM8K Qwen experiment provides a complementary language-reasoning evaluation of the same multi-rollout idea.

The relatively high accuracy in Figure 14 is expected for several reasons. First, `Qwen2.5-Math-7B-Instruct` is a math-specialized instruction-tuned model rather than a small general-purpose language model. It has been optimized for mathematical reasoning tasks, so GSM8K is much closer to its intended use case than it would be for a base model such as `distilgpt2`. Second, our experiment evaluates inference-time sampling from a pretrained model rather than training a model from scratch. Therefore, the high pass@1 score reflects the strong prior reasoning ability already present in the pretrained Qwen model.

In addition, our evaluation is performed on a limited GSM8K test subset rather than a full benchmark sweep. A smaller subset can produce higher or lower accuracy depending on the difficulty of the selected questions. Therefore, the absolute accuracy values should be interpreted as a small-scale rollout-sampling study rather than a definitive benchmark result. The main point of the experiment is the trend that pass@k increases as the number of sampled reasoning rollouts increases.

Another reason pass@k is high is that pass@k is an oracle-style metric. A problem is counted as correct if any one of the k generated solutions gives the right final answer. Therefore, pass@4 and pass@8 naturally exceed pass@1 because the model is given more chances to sample a correct reasoning path. This does not mean the system can always identify the correct solution without access to the ground truth. For that reason, we also report majority-vote@k as a more realistic non-oracle aggregation method.

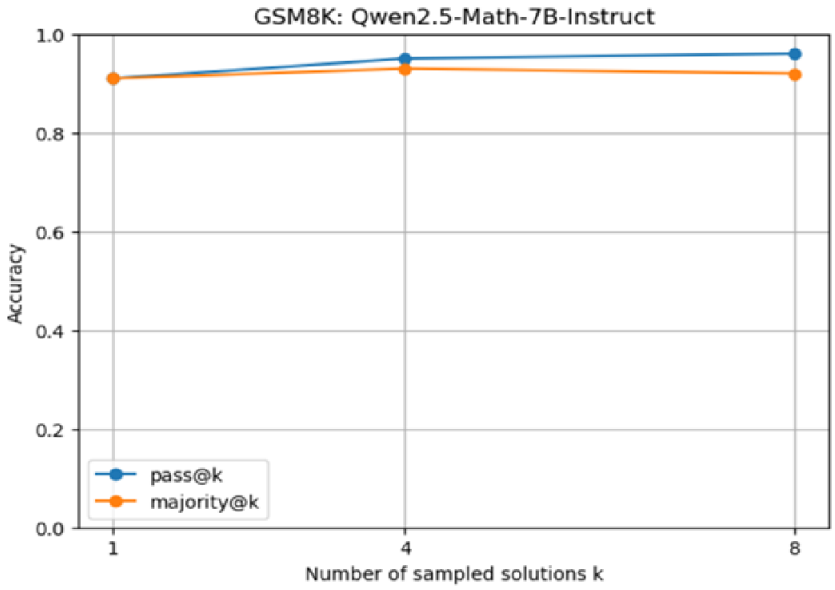

Figure 14: GSM8K inference-time rollout experiment using Qwen2.5-Math-7B-Instruct